Faculty of Engineering Ain Shams University

CSE471s: Artificial Intelligence – Spring 2026

Quoridor Game Implementation

Project Report

ID	Name
2300189	Adam Ahmed Abdelhamid
2300191	Adham Khaled Mohamed
2300163	Abdelrahman Zakaria Talaat
2300360	Lobna Ahmed Abd Elhamid
2300429	Malak Mohammed Ali

Github Repo:
https://github.com/Adam8922/play_quoridor_versus_ai.git

Demo Video:
https://drive.google.com/drive/folders/1hJG4wIEBOWpDmvUi0zTU4cX_s7b8V1Mp?usp=drive_link

Quoridor Strategic AI Implementation 2
1. Project Introduction 2
2. System Architecture & Design 2
A. Modular File Structure 2
B. Core Data Structures 2
3. Detailed Implementation of Game Logic 3
A. Representing Walls (Edge Matrices) 3
B. Pawn Movement Validation 3
C. The "No-Trapping" Referee 3
4. AI & Pathfinding Implementation 3
A. Pawn-Aware BFS 3
B. Multi-Tiered AI Difficulties 4
C. Strategic Scoring & Move Ordering 4
5. Technical Challenges & Optimization 4
Optimization: The "Undo" Revolution 4
Threading for a Smooth UI 5
Dynamic Resolution Scaling 5
6. Resources Used 5

Note: Undo/Redo bonus applied

Quoridor Strategic AI Implementation

1. Project Introduction

In this project, We developed a digital implementation of the strategy board game **Quoridor**. The game is played on a 9x9 grid where two players compete to reach the opposite side first. Players can either move their pawn or place a wall to block their opponent.

The goal of this project was to create a game that is not only fun to play but also features a "Hard" AI that can challenge even experienced players. We focused on building a robust game engine and a fast decision-making AI using modern search algorithms.

---

2. System Architecture & Design

We designed the project using a modular structure to keep the logic separated from the graphics. This "separation of concerns" made debugging much easier.

A. Modular File Structure

- **`main.py`**: The entry point. It manages the high-level state (Menu vs. Gameplay) and handles the main loop at 60 frames per second.
- **`Board.py` (The Engine)**: This is the core logic. It acts as a stateless engine, intentionally not tracking turns so it can be freely manipulated by AI algorithms. It manages pawn positions, wall counts, and move history using an undo/redo ledger.
- **`AIPlayer.py` (The Brain)**: Contains the specialized algorithms for pathfinding and strategic decision-making with varying difficulty levels.
- **`game_screen.py` & `menu_screen.py`**: Handle visual rendering using Pygame, with a dynamic resizing system to support different screen resolutions.
- **`Constants.py`**: A central location for all "Magic Numbers" like colors (Neon Red/Blue), cell sizes (50px), and margins.

B. Core Data Structures

We used Python **Dataclasses** (`Position`, `Wall`, `GameMove`) to keep the data organized. For example, a `Wall` consists of a `top_left` position and an `orientation` (Horizontal or Vertical). Moves are tracked using `GameMove` containing previous states and move types, creating an immutable history suitable for undo/redo logic.

3. Detailed Implementation of Game Logic

The most critical part of a Quoridor game is ensuring that the rules are followed strictly.

A. Representing Walls (Edge Matrices)

Instead of just drawing lines, We used two 2D lists (matrices) to represent the physical space between cells:

- `_horizontal_edges` (8x9 matrix): Tracks walls between rows.
- `_vertical_edges` (9x8 matrix): Tracks walls between columns.
- `_crosses_blocked` (8x8 matrix): A boolean matrix locking the "Top-Left Peg" to prevent placing a vertical wall directly crossing a horizontal wall.
  This allows the engine to check if a move is blocked in $O(1)$ time—basically instant.

B. Pawn Movement Validation

Movement validation includes comprehensive checks:

- **Normal Moves**: 1 step checked against edge matrices.
- **Straight Jumps**: Verifies opponent adjacency and clear landing paths.
- **Diagonal Jumps (Anti-Teleport Lock)**: Only permitted if the straight jump path is blocked by a wall or the board edge, preventing illegal "teleporting" slides.

C. The "No-Trapping" Referee

A major rule in Quoridor is that you cannot completely block an opponent. Our code acts as a "Referee": every time a wall is proposed, it runs a **Breadth-First Search (BFS)** for both players. If the proposed wall makes the shortest path length zero (meaning no path exists), the move is rejected.

4. AI & Pathfinding Implementation

A. Pawn-Aware BFS

A standard maze-solving BFS isn't enough for Quoridor because players can "jump" over each other. We implemented a **Specialized BFS** (`get_shortest_path_with_pawns`) in `AIPlayer.py` that calculates neighbors based on the current positions of both pawns. It correctly identifies straight jumps and diagonal jumps, which is essential for the AI to calculate the *true* shortest path.

B. Multi-Tiered AI Difficulties

- **Easy Mode ("Lazy Greedy")**: The Easy AI is designed to be beatable by making sub-optimal choices:
  - Movement (80% chance): It calculates the shortest path to its goal and takes the first step along that path.
  - Walls (20% chance): It has a small chance to place a random valid wall near the opponent's current path, rather than trying to block them strategically.
- **Medium Mode ("Greedy")**: The Medium AI focuses on immediate advantage without looking ahead to future turns:
  - Offense: If it is already winning (its path to the goal is shorter than or equal to the opponent's), it simply moves toward the goal.
  - Defense: If it is losing, it searches for a wall placement that provides the maximal immediate gain (the wall that lengthens the opponent's shortest path by the most squares).
  - Fallback: If no beneficial wall can be placed, it moves toward the goal.
- **Hard Mode (Minimax & Alpha-Beta)**: The Hard AI uses a classic adversarial search algorithm to simulate future moves:
  - Algorithm: Minimax with a search depth of 3 plies (3 turns ahead).
  - Optimization: Uses Alpha-Beta Pruning to discard move branches that cannot possibly affect the final decision, significantly improving performance.
  - Heuristic Evaluation: It evaluates board states using a formula: Score = (Opponent Path Length - My Path Length) + (My Walls - Opponent Walls) * 0.1
    - This prioritizes shortening its own path while lengthening the opponent's, with a small bonus for keeping more walls in reserve.
  - Move Ordering: It sorts moves to prioritize path-advancing pawn moves and the most disruptive wall placements first, which makes Alpha-Beta pruning more efficient.

C. Strategic Scoring & Move Ordering

To decide which move is best, the Hard AI uses a scoring function:

- **Primary Goal**: Minimize its own distance to the goal and maximize the opponent's distance (using the pawn-aware BFS).
- **Secondary Goal**: Save walls for the endgame. Each wall left is worth a small bonus (0.1 points).

- **Candidate Filtering**: To keep the search fast, the AI prioritizes path-advancing pawn moves and only considers the "best" 20 wall placements—those that are within 2 squares of the opponent's current path, scored initially by how much they delay the opponent.

---

5. Technical Challenges & Optimization

Optimization: The "Undo" Revolution

Originally, our AI was very slow because it used `deepcopy` to clone the entire board thousands of times per turn. This made "Hard" mode unplayable.

- **The Solution**: We implemented an **In-Place Mutation and Undo** system. The AI now makes a move, evaluates it, and then instantly reverses the move using `undo_data` (restoring pawn positions and wall counts). This change reduced the AI's "thinking time" from **10+ seconds to under 2 seconds**.

Threading for a Smooth UI

Calculations in "Hard" mode can be heavy. To prevent the Pygame window from freezing ("Not Responding"), We moved the AI logic to a **background thread**. This allows the main thread to continue drawing the screen, processing hover states, and showing a "Thinking..." status message while the background thread does the heavy lifting.

Dynamic Resolution Scaling

To ensure the game is playable on all monitors, `game_screen.py` implements a dynamic window resizing algorithm. The board and UI elements calculate their scale and margins dynamically based on the current window dimensions rather than using hardcoded pixel values.

6. Resources Used

Beside the project description PDF helpful resources:

- **Pygame Community**: For graphics and event handling tutorials.
- **GeeksforGeeks**: For detailed explanations of Minimax and Alpha-Beta Pruning.
- **Gigamic Official Rules**: To ensure jump and wall placement logic was 100% accurate.